FINAL CLIENT PRESENTATION

BASTION

BASE LOGISTICS & REQUISITION COMMAND

One secure system for every depot, every item and every requisition on base, from request to issue to return.



Developer: [Your Full Name]
Delivered to: the Supply Office · React Fundamentals with Laravel Backend

OPENING

The problem we set out to solve

TODAY

- Requests live on paper and spreadsheets
- Stock counts drift out of date
- Approvals get lost between offices
- Nobody can say who holds which equipment

GOALS FOR THIS SYSTEM

- One live record of every item, by depot
- A clear request → approve → issue → return trail
- Low-stock warnings before shortages hurt
- Right people see and do the right things

SOLUTION OVERVIEW

Two roles, one workflow

SUPPLY OFFICER

- Registers equipment
- Files requisitions, edits while pending
- Tracks their own requests

BASE COMMANDER

- Approves, rejects, issues, closes out
- Sees every request and the personnel roster
- Manages equipment status and deletions

A REQUISITION'S LIFE

PENDING → APPROVED → ISSUED → RETURNED or REJECTED

Issuing takes units out of stock; returning puts them back. Everything also works on a phone.

SOLUTION OVERVIEW

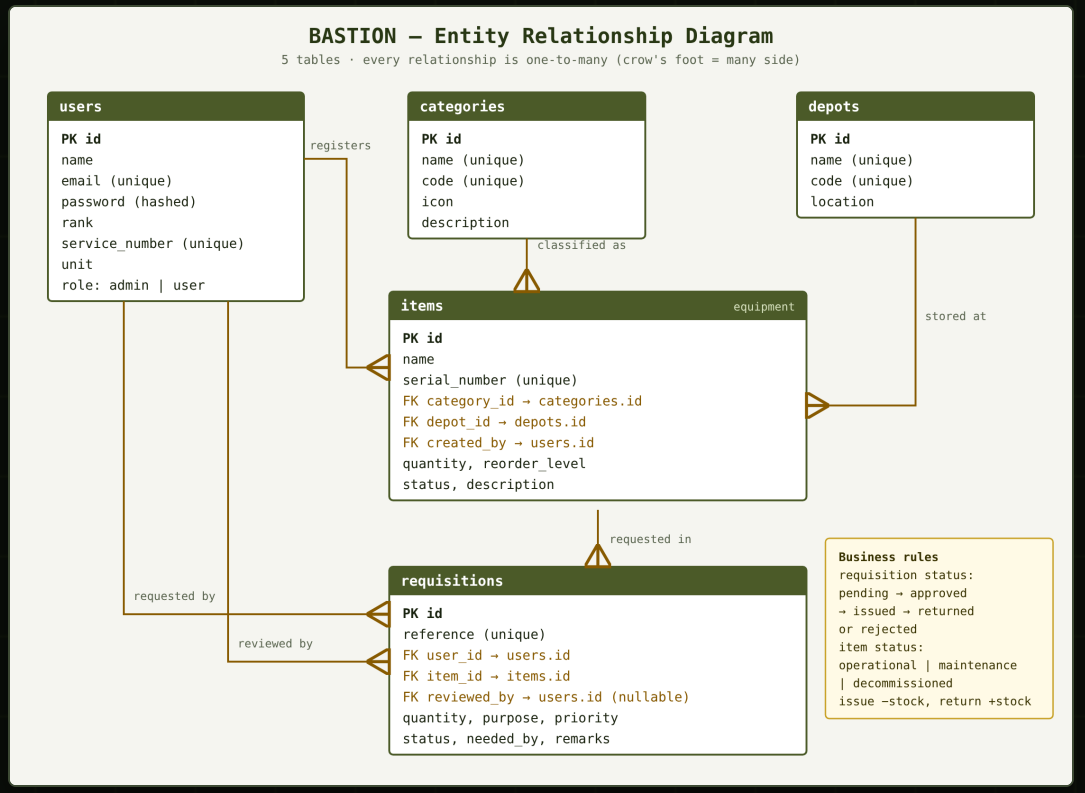
How it works, in plain language

- **The screen** (React) is what people see and click
- **The rules desk** (Laravel API) checks who you are and what you may do
- **The filing cabinet** (MySQL) keeps every record

Sign-in gives each person a private token. Every request carries it, so the system always knows who is asking and what they are allowed to do.

5 TABLES

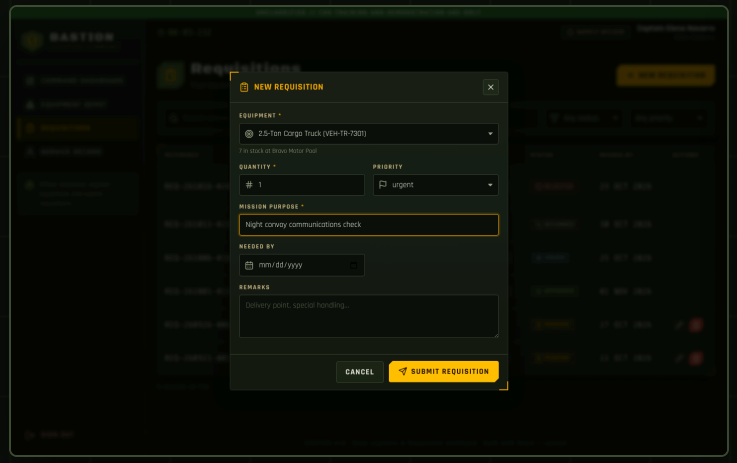
Users · Categories · Depots · Items · Requisitions. An item belongs to one category and one depot; a requisition links an officer to an item.



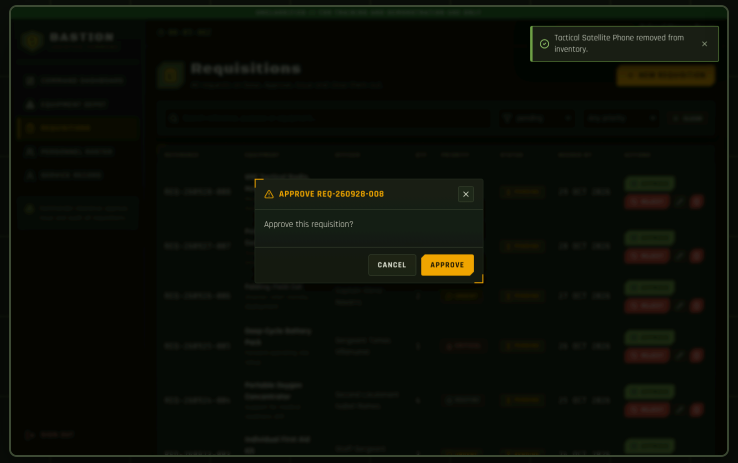
Entity relationship diagram

LIVE PRODUCT WALKTHROUGH

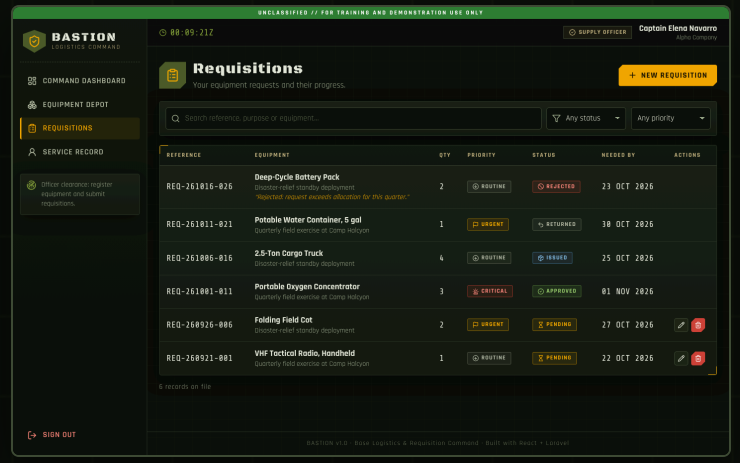
Scenario: a night-convoy radio request



1. Officer files the request (validation protects the form)



2. Commander reviews and approves, then issues



3. Officer sees the update; stock is adjusted

WATCH FOR

- Clear red messages when a form is wrong
- A confirmation before every approval or delete

ALSO SHOWN LIVE

- Search, filters and pagination
- Dashboard figures and role restrictions

HANDOVER

Install it, run it, keep it secure

REPOSITORY & SETUP

Repo: [GitHub repo link]

```
cd backend
composer install
php artisan migrate --seed
php artisan serve
cd ../frontend
npm install && npm run dev
```

Full steps and 31-request Postman collection are in the README and `/docs`.

DEMO ACCOUNTS

ROLE	EMAIL
Commander	admin@bastion.test
Officer	officer@bastion.test

Password for both: `Password123!`

HOW IT IS SECURED

- Token sign-in; passwords stored hashed
- Admin pages blocked in the screen **and** the API
- Ownership rules: officers touch only their own
- Validation on both sides; login rate-limited

CLOSING

What was delivered

- ✔ Register, sign in, sign out, current user
- ✔ Two roles with enforced access
- ✔ Full CRUD: equipment and requisitions
- ✔ Search and filters on both lists
- ✔ Pagination on every list
- ✔ Dashboard: 8 figures + 2 charts
- ✔ Validation, loaders, toasts, confirmations
- ✔ Approval workflow with stock tracking

Beyond the brief: personnel roster (commander only), low-stock alerts, equipment status control, mobile layout, automated API tests.

24	28	6	31
equipment items seeded	requisitions in every state	demo accounts, 2 roles	documented API requests

CLOSING

Limitations and Phase 2

KNOWN LIMITATIONS

- No email or SMS notifications yet
- No export of reports
- No equipment photos
- One shared stock count per item (not per depot)

RECOMMENDED PHASE 2

- Email alerts for approvals and low stock
- PDF / CSV reports
- Photo upload per item
- Stock transfers between depots
- Audit log of every change

Thank you, Client. BASTION is ready for handover.